Vitalia: Contrato de la API v1

Nov 2, 2026 · @Alexis Hernandez

Propósito y alcance

Este documento es el acuerdo oficial entre los cuatro equipos sobre cómo se comunican el front y el backend de Vitalia. Si algo no está aquí, no existe en la API; si alguien necesita un cambio, se pide por el proceso de la última sección.

Vitalia es un directorio web de veterinarias: los dueños de mascotas buscan, comparan y contactan clínicas cercanas, y las clínicas ganan visibilidad. La API cubre búsqueda por ubicación, detalle de clínicas, reseñas, carnet de mascota, comunidad, panel de negocio y administración.

Dato	Valor
Versión del contrato	1.3
Versión de la API	v1
Tecnología del backend	Python 3.12+, FastAPI, SQLite
Documentación interactiva	/docs en el servidor del backend

Equipo	Qué toma de este contrato
Frontend	Rutas, parámetros, formato de respuestas y errores
Backend	Lo implementa tal cual y lo mantiene actualizado
Base de datos	Los modelos de datos que deben existir en las tablas
Infraestructura	Variables de entorno, puertos y endpoint de salud

Fuera de alcance en v1: pagos en línea, citas agendadas, chat interno y app móvil nativa.

Convenciones generales

Todas las rutas cuelgan de `/api/v1` y hablan solo JSON en UTF-8.

Tema	Regla	Ejemplo
URL base local	`http://localhost:8000/api/v1`	`GET http://localhost:8000/api/v1/salud`
Versionado	En la ruta; un cambio que rompe compatibilidad crea `/api/v2`	`/api/v1/clinicas`
Nombres de rutas	Español, minúsculas, plural, guion medio	`/me/mascotas`, `/mensaje-whatsapp`
Campos JSON	camelCase	`totalResenas`, `urgencias24h`
Columnas en BD	snake_case; el backend hace la conversión	`total_resenas`
IDs	Enteros positivos	`"id": 12`
Fechas con hora	ISO 8601 con zona horaria	`"2026-11-02T18:30:00-06:00"`
Solo hora	24 horas, `HH:MM`	`"21:00"`
Zona horaria de negocio	America/Mexico_City; con ella se calcula "abierto ahora"	
Dinero	Número en pesos mexicanos, dos decimales	`"precio": 350.00`
Coordenadas	Grados decimales, `lat` y `lng` separados	`"lat": 19.3801, "lng": -99.1702`
Distancias	Kilómetros, un decimal	`"distanciaKm": 0.6`
Campos vacíos	Se envían como `null`, nunca se omiten	`"fotoUrl": null`
Booleanos	`true` o `false`; en query string `true` o `false`	`?abiertoAhora=true`

Encabezados

Encabezado	Cuándo	Valor
`Content-Type`	Peticiones con cuerpo JSON	`application/json`
`Content-Type`	Subida de fotos	`multipart/form-data`
`Authorization`	Rutas protegidas	`Bearer <token>`

Paginación

Toda lista que pueda crecer se pagina con `pagina` y `porPagina`. Por defecto `pagina=1` y `porPagina=10`; el máximo de `porPagina` es 50. La respuesta trae siempre el bloque `meta`:

```
{ "pagina": 1, "porPagina": 10, "total": 37, "totalPaginas": 4 }
```

Límites

- Fotos: JPG, PNG o WebP, máximo 5 MB cada una, máximo 10 por clínica.
- Textos largos como comentarios de reseña: máximo 1000 caracteres.
- Radio de búsqueda: de 1 a 50 km.

Autenticación y roles

La API usa un token JWT que dura 8 horas; no hay refresh token en v1, así que al vencer el usuario vuelve a iniciar sesión.

1. El usuario se registra con `POST /auth/registro` o entra con `POST /auth/login`.
2. La respuesta trae `token` y los datos del usuario.
3. El front guarda el token y lo manda en cada petición protegida como `Authorization: Bearer <token>`.
4. Si la API responde 401 con código `TOKEN_INVALIDO` o `NO_AUTENTICADO`, el front borra el token y manda al login.
5. Cerrar sesión es solo borrar el token en el front.

El token lleva dentro el `id` del usuario, su `rol` y la fecha de vencimiento. El front puede leer el rol para mostrar u ocultar menús, pero la API siempre vuelve a validar el permiso.

Las contraseñas se guardan con hash bcrypt y deben tener mínimo 8 caracteres, con al menos una letra y un número.

Roles

Rol	Valor en la API	Cómo se obtiene	Qué puede hacer
Visitante	sin token	No inicia sesión	Buscar clínicas, ver detalle, reseñas, fotos, categorías y leer la comunidad
Dueño	`dueno`	Registro normal	Todo lo del visitante, más reseñar, carnet de mascotas y publicar en la comunidad
Clínica	`clinica`	Registro desde "Para negocios"	Todo lo del visitante, más registrar y administrar sus clínicas y publicar en la comunidad
Admin	`admin`	Lo crea el equipo directo en la BD	Verificar clínicas y moderar reseñas y la comunidad

Una cuenta de clínica solo puede editar las clínicas donde es propietaria. Si intenta tocar otra, la API responde 403 `SIN_PERMISO`.

Formato de respuestas y errores

Toda respuesta trae `ok`; el front decide con ese campo si leer `data` o `error`.

Respuesta exitosa de un objeto:

```
{ "ok": true, "data": { "id": 12, "nombre": "Clínica Veterinaria Patitas" } }
```

Respuesta exitosa de una lista paginada:

```
{
  "ok": true,
  "data": [ { "id": 12 }, { "id": 15 } ],
  "meta": { "pagina": 1, "porPagina": 10, "total": 2, "totalPaginas": 1 }
}
```

Respuesta de error; `detalles` solo aparece en errores de validación:

```
{
  "ok": false,
  "error": {
    "codigo": "VALIDACION",
    "mensaje": "Los datos enviados no son válidos",
    "detalles": [ { "campo": "email", "mensaje": "Formato de correo inválido"
} ]
  }
}
```

El front debe guiarse por `codigo`, que es fijo. El `mensaje` es para mostrar y puede cambiar de redacción.

Catálogo de errores

HTTP	Código	Cuándo pasa
400	`VALIDACION`	Faltan campos, tipos incorrectos o valores fuera de rango
400	`ARCHIVO_INVALIDO`	La foto no es JPG, PNG o WebP, o pesa más de 5 MB
401	`NO_AUTENTICADO`	La ruta pide token y no llegó
401	`TOKEN_INVALIDO`	El token está vencido o alterado
401	`CREDENCIALES_INVALIDAS`	Correo o contraseña incorrectos en el login
403	`SIN_PERMISO`	El rol no alcanza o el recurso es de otro usuario
404	`NO_ENCONTRADO`	El recurso o la ruta no existe
405	`METODO_NO_PERMITIDO`	Se usó un método HTTP que la ruta no acepta
409	`CONFLICTO`	Correo ya registrado o reseña repetida
413	`ARCHIVO_MUY_GRANDE`	El cuerpo de la petición supera el límite del servidor
429	`DEMASIADAS_PETICIONES`	Se excedió el límite de intentos, por ejemplo en login
500	`ERROR_INTERNO`	Falla no controlada del servidor

Códigos de éxito

HTTP	Uso
200	Lectura o edición correcta
201	Recurso creado; `data` trae el objeto nuevo
204	Borrado correcto; no trae cuerpo

Modelos de datos

Estos son los objetos que viajan en la API; los endpoints los nombran por su título. Un campo marcado "puede ser null" siempre viene, aunque sea con `null`.

Valores fijos

Campo	Valores permitidos
`rol`	`dueno`, `clinica`, `admin`
`especie`	`perro`, `gato`, `ave`, `conejo`, `reptil`, `otro`
`sexo`	`macho`, `hembra`
`tipos` de clínica	`veterinaria`, `hospital`, `estetica`, `hospedaje`, `adiestramiento`
`diaSemana`	1 lunes, 2 martes, 3 miércoles, 4 jueves, 5 viernes, 6 sábado, 7 domingo
`orden` en búsqueda	`distancia`, `calificacion`, `resenas`

Usuario

Campo	Tipo	Ejemplo	Nota
`id`	entero	`7`	
`nombre`	texto	`"Rosa Martínez"`	
`email`	texto	`"rosa@mail.com"`	Único

Campo	Tipo	Ejemplo	Nota
`telefono`	texto	`"5512345678"`	Puede ser null; 10 dígitos
`rol`	texto	`"dueno"`	Ver valores fijos
`creadoEn`	fecha y hora	`"2026-11-02T18:30:00-06:00"`	

La contraseña nunca viaja de regreso en ninguna respuesta.

Mascota

Es el carnet de la pantalla "Carnet de mi mascota".

Campo	Tipo	Ejemplo	Nota
`id`	entero	`3`	
`nombre`	texto	`"Luna"`	Máximo 50 caracteres
`especie`	texto	`"perro"`	Ver valores fijos
`raza`	texto	`"Golden Retriever"`	Puede ser null
`sexo`	texto	`"hembra"`	
`edadAnios`	entero	`3`	Puede ser null; de 0 a 40
`edadMeses`	entero	`0`	Puede ser null; de 0 a 11, para cachorros
`color`	texto	`"Dorado"`	Puede ser null
`fotoUrl`	texto	`"https://.../luna.jpg"`	Puede ser null

Categoría

Alimenta "Servicios más buscados" y agrupa los servicios de cada clínica.

Campo	Tipo	Ejemplo
`id`	entero	`2`
`nombre`	texto	`"Vacunas"`

Campo	Tipo	Ejemplo
`slug`	texto	`"vacunas"`
`icono`	texto	`"syringe"`

Categorías iniciales: consulta-general, urgencias, vacunas, cirugia, desparasitacion, rayos-x, estetica, hospedaje, adiestramiento, hospitalizacion. El icono es un nombre que el front traduce a su propio ícono.

ClínicaResumen

Es lo que pinta cada tarjeta de "Veterinarias cerca de ti" y cada fila de la lista del mapa.

Campo	Tipo	Ejemplo	Nota
`id`	entero	`12`	
`nombre`	texto	`"Clínica Veterinaria Patitas"`	
`tipos`	lista de texto	`["veterinaria"]`	Ver valores fijos
`calificacion`	número	`4.8`	Puede ser null si no tiene reseñas; un decimal
`totalResenas`	entero	`212`	
`distanciaKm`	número	`0.6`	Null si la búsqueda no mandó ubicación
`abierto`	booleano	`true`	Calculado con la hora de CDMX
`cierraA`	texto	`"21:00"`	Null si está cerrada o es 24 h
`abreA`	texto	`"09:00"`	Solo si está cerrada; siguiente apertura

Campo	Tipo	Ejemplo	Nota
`urgencias24h`	booleano	`false`	Pinta la etiqueta roja
`verificada`	booleano	`true`	Revisada por un admin
`fotoPrincipal`	texto	`"https://.../patitas.jpg"`	Puede ser null
`serviciosDestacados`	lista de texto	`["Consulta", "Vacunas", "Cirugía"]`	Máximo 3
`lat`	número	`19.3801`	
`lng`	número	`-99.1702`	

ClínicaDetalle

Incluye todos los campos de ClínicaResumen, más estos:

Campo	Tipo	Ejemplo	Nota
`descripcion`	texto	`"Atención integral para perros y gatos"`	Puede ser null
`direccion`	objeto	ver abajo	
`telefono`	texto	`"5598765432"`	Para el botón Llamar
`whatsapp`	texto	`"5215598765432"`	Puede ser null; con lada de país
`sitioWeb`	texto	`"https://patitas.mx"`	Puede ser null
`horarios`	lista de Horario		Vacía si es 24 h
`fotos`	lista de Foto		Las primeras 5
`servicios`	lista de Servicio		Todos

`direccion` lleva `calle`, `colonia`, `alcaldiaMunicipio`, `ciudad`, `estado` y `codigoPostal`, todos texto, más `textoCompleto` ya armado para mostrar.

El botón "Cómo llegar" lo arma el front con `lat` y `lng`; "Compartir" usa la URL pública del front con el `id`.

Horario

Campo	Tipo	Ejemplo	Nota
`diaSemana`	entero	`1`	Ver valores fijos
`abre`	texto	`"09:00"`	
`cierra`	texto	`"21:00"`	Mayor que `abre`

Un día puede tener varios horarios, por ejemplo de 09:00 a 14:00 y de 16:00 a 20:00. Un día sin horarios significa cerrado.

Servicio

Campo	Tipo	Ejemplo	Nota
`id`	entero	`40`	
`categoria`	Categoría		
`nombre`	texto	`"Cirugía"`	
`descripcion`	texto		Puede ser null
`precio`	número	`2000.00`	Puede ser null si es "precio a consultar"
`esPrecioDesde`	booleano	`true`	Si es `true`, el front muestra "Desde $2,000"

Reseña

Campo	Tipo	Ejemplo	Nota
`id`	entero	`501`	
`calificacion`	entero	`5`	De 1 a 5
`comentario`	texto	`"Muy buena atención"`	Puede ser null; máximo 1000 caracteres

Campo	Tipo	Ejemplo	Nota
`autor`	objeto	`{ "id": 7, "nombre": "Rosa M." }`	Solo nombre e inicial del apellido
`creadoEn`	fecha y hora		
`editadoEn`	fecha y hora		Null si nunca se editó
`esMia`	booleano	`true`	Null sin sesión; sirve para mostrar Editar y Borrar

Foto

Campo	Tipo	Ejemplo
`id`	entero	`88`
`url`	texto	`"https://.../foto.jpg"`
`orden`	entero	`1`

Publicación

Es una entrada de la sección Comunidad.

Campo	Tipo	Ejemplo	Nota
`id`	entero	`31`	
`autor`	objeto	`{ "id": 7, "nombre": "Rosa M." }`	Nombre e inicial del apellido; si la cuenta se borró, `id` es null y el nombre es "Usuario eliminado"
`texto`	texto	`"Luna ya tiene sus vacunas"`	De 1 a 2000 caracteres
`fotos`	lista de Foto		Máximo 4; vacía hasta que exista la subida de fotos

Campo	Tipo	Ejemplo	Nota
`totalComentarios`	entero	`3`	
`esMia`	booleano	`true`	Null sin sesión; sirve para mostrar Editar y Borrar
`creadoEn`	fecha y hora		
`editadoEn`	fecha y hora		Null si nunca se editó

Comentario

Campo	Tipo	Ejemplo	Nota
`id`	entero	`120`	
`autor`	objeto	`{ "id": 9, "nombre": "Luis" }`	Igual que en Publicación
`texto`	texto	`"¿Dónde la vacunaste?"`	De 1 a 1000 caracteres
`esMio`	booleano	`false`	Null sin sesión
`creadoEn`	fecha y hora		

Endpoints de autenticación y cuenta

Método	Ruta	Rol	Para qué
POST	`/auth/registro`	Público	Crear cuenta de dueño o de clínica
POST	`/auth/login`	Público	Iniciar sesión
GET	`/auth/me`	Logueado	Datos del usuario actual
PATCH	`/me`	Logueado	Editar nombre o teléfono
PUT	`/me/password`	Logueado	Cambiar contraseña
DELETE	`/me`	Logueado	Borrar la cuenta

Método	Ruta	Rol	Para qué
GET	`/salud`	Público	Revisar que API y BD respondan

POST /auth/registro

`rol` solo acepta `dueno` o `clinica`; el botón Regístrate manda `dueno` y la sección "Para negocios" manda `clinica`.

```
{
  "nombre": "Rosa Martínez",
  "email": "rosa@mail.com",
  "password": "Perrito2026",
  "telefono": "5512345678",
  "rol": "dueno"
}
```

Respuesta 201:

```
{
  "ok": true,
  "data": {
    "token": "eyJhbGciOi...",
    "expiraEn": "2026-11-03T02:30:00-06:00",
    "usuario": { "id": 7, "nombre": "Rosa Martínez", "email":
"rosa@mail.com", "telefono": "5512345678", "rol": "dueno", "creadoEn": "2026-
11-02T18:30:00-06:00" }
  }
}
```

Errores: 400 `VALIDACION`, 409 `CONFLICTO` si el correo ya existe.

POST /auth/login

```
{ "email": "rosa@mail.com", "password": "Perrito2026" }
```

Respuesta 200: igual que el registro.

Errores: 401 `CREDENCIALES_INVALIDAS` sin decir si falló el correo o la contraseña; 429 `DEMASIADAS_PETICIONES` tras 5 intentos fallidos en 15 minutos desde la misma IP.

GET /auth/me

Respuesta 200: `data` es un Usuario. Error 401 si no hay token o está vencido.

PATCH /me

Manda solo los campos a cambiar. El correo y el rol no se pueden editar en v1.

```
{ "nombre": "Rosa M. López", "telefono": "5511112222" }
```

Respuesta 200: el Usuario actualizado.

PUT /me/password

```
{ "passwordActual": "Perrito2026", "passwordNueva": "Gatito2027" }
```

Respuesta 204. Errores: 401 `CREDENCIALES_INVALIDAS` si la actual no coincide; 400 `VALIDACION` si la nueva no cumple las reglas.

DELETE /me

Borra la cuenta y sus mascotas. Sus reseñas, publicaciones y comentarios se quedan pero el autor aparece como "Usuario eliminado". Una cuenta de clínica con clínicas registradas recibe 409 `CONFLICTO` y primero debe darlas de baja.

Respuesta 204.

GET /salud

Lo usa infraestructura para monitoreo. Respuesta 200:

```
{ "ok": true, "data": { "api": "funcionando", "baseDatos": "conectada",
"entorno": "local", "version": "0.1.0" } }
```

Endpoints públicos de clínicas y búsqueda

No piden token.

Método	Ruta	Para qué	Pantalla
GET	`/clinicas`	Buscar y filtrar clínicas	Inicio, mapa
GET	`/clinicas/{id}`	Detalle completo	Detalle
GET	`/clinicas/{id}/servicios`	Servicios con precio	Pestaña Servicios
GET	`/clinicas/{id}/resenas`	Reseñas paginadas	Pestaña Reseñas
GET	`/clinicas/{id}/fotos`	Galería completa	Pestaña Fotos
GET	`/categorias`	Catálogo de categorías	Servicios más buscados, filtros

GET /clinicas

Es el endpoint central. Lo usan la caja de búsqueda, los filtros rápidos, "Veterinarias cerca de ti", la lista del mapa y el botón de Urgencias.

El backend no convierte direcciones en coordenadas: el front usa la geolocalización del navegador o el buscador de su librería de mapas y manda `lat` y `lng`.

Parámetro	Tipo	Por defecto	Regla
`lat`	número		De -90 a 90; va junto con `lng`
`lng`	número		De -180 a 180; va junto con `lat`
`radioKm`	número	5	De 1 a 50; solo aplica con ubicación
`q`	texto		Busca en nombre y colonia; mínimo 2 letras
`tipo`	texto		Uno de los tipos de clínica
`categoria`	texto		Slug de categoría, por ejemplo `vacunas`

Parámetro	Tipo	Por defecto	Regla
`abiertoAhora`	booleano	false	Solo las abiertas en este momento
`urgencias`	booleano	false	Solo las de urgencias 24 h
`orden`	texto	`distancia` con ubicación, `calificacion` sin ella	`distancia`, `calificacion` o `resenas`
`pagina`	entero	1	
`porPagina`	entero	10	Máximo 50; el mapa puede pedir 50

Cómo se mapean los chips del mockup:

Chip	Parámetro
Abierto ahora	`abiertoAhora=true`
Urgencias 24 h	`urgencias=true`
Veterinarias	`tipo=veterinaria`
Estéticas	`tipo=estetica`
Hospedaje	`tipo=hospedaje`
Adiestramiento	`tipo=adiestramiento`
Íconos de Servicios más buscados	`categoria=<slug>`

Ejemplo: `GET /clinicas?lat=19.3801&lng=-99.1702&radioKm=5&abiertoAhora=true&pagina=1`

Respuesta 200: `data` es una lista de ClínicaResumen y `meta` agrega la búsqueda usada, para pintar el círculo del mapa y el texto "5 veterinarias cerca de ti".

```
{
  "ok": true,
  "data": [
    {
      "id": 12,
      "nombre": "Clínica Veterinaria Patitas",
```

```
      "tipos": ["veterinaria"],
      "calificacion": 4.8,
      "totalResenas": 212,
      "distanciaKm": 0.6,
      "abierto": true,
      "cierraA": "21:00",
      "abreA": null,
      "urgencias24h": false,
      "verificada": true,
      "fotoPrincipal": "https://.../patitas.jpg",
      "serviciosDestacados": ["Consulta", "Vacunas", "Cirugía"],
      "lat": 19.3801,
      "lng": -99.1702
    }
  ],
  "meta": {
    "pagina": 1, "porPagina": 10, "total": 5, "totalPaginas": 1,
    "busqueda": { "lat": 19.3801, "lng": -99.1702, "radioKm": 5 }
  }
}
```

Solo aparecen clínicas activas. Las no verificadas sí salen, pero sin la marca de verificada.

Errores: 400 `VALIDACION` si viene `lat` sin `lng`, si el radio está fuera de rango o si piden `orden=distancia` sin ubicación.

GET /clinicas/{id}

Acepta `lat` y `lng` opcionales para calcular `distanciaKm`, que en el detalle se muestra como "0.6 km de tu ubicación".

Respuesta 200: `data` es una ClínicaDetalle. Error 404 `NO_ENCONTRADO` si no existe o está dada de baja.

GET /clinicas/{id}/servicios

Respuesta 200: lista de Servicio ordenada por categoría y nombre, sin paginar.

GET /clinicas/{id}/resenas

Parámetros: `pagina`, `porPagina` y `orden`, que acepta `recientes`, por defecto, `mejores` o `peores`.

Respuesta 200: lista de Reseña. `meta` agrega el resumen para la cabecera de la pestaña:

```
"resumen": { "promedio": 4.8, "total": 212, "distribucion": { "5": 170, "4": 30, "3": 8, "2": 3, "1": 1 } }
```

GET /clinicas/{id}/fotos

Respuesta 200: lista de Foto ordenada por `orden`.

GET /categorias

Respuesta 200: lista de Categoría, sin paginar. Cambia poco, así que el front puede guardarla en caché durante la sesión.

Endpoints del dueño

Todos piden token con rol `dueno`; un usuario solo ve y toca sus propias mascotas y reseñas.

Método	Ruta	Para qué
POST	`/clinicas/{id}/resenas`	Dejar una reseña
PUT	`/resenas/{id}`	Editar mi reseña
DELETE	`/resenas/{id}`	Borrar mi reseña
GET	`/me/mascotas`	Mis mascotas
GET	`/me/mascotas/{id}`	Una mascota
POST	`/me/mascotas`	Crear carnet
PUT	`/me/mascotas/{id}`	Editar carnet
DELETE	`/me/mascotas/{id}`	Borrar carnet
POST	`/me/mascotas/{id}/foto`	Subir o cambiar foto
GET	`/me/mascotas/{id}/mensaje-whatsapp`	Texto y link de WhatsApp con el carnet

POST /clinicas/{id}/resenas

```
{ "calificacion": 5, "comentario": "Atendieron rápido a mi perrita" }
```

Respuesta 201: la Reseña creada. La API recalcula `calificacion` y `totalResenas` de la clínica.

Errores: 404 si la clínica no existe; 409 `CONFLICTO` si ya tiene una reseña en esa clínica, en ese caso se edita la existente.

PUT /resenas/{id} y DELETE /resenas/{id}

PUT recibe el mismo cuerpo que el POST y responde 200 con la Reseña. DELETE responde 204. Si la reseña es de otro usuario: 403 `SIN_PERMISO`.

Mascotas

`POST /me/mascotas` y `PUT /me/mascotas/{id}` reciben:

```
{
  "nombre": "Luna",
  "especie": "perro",
  "raza": "Golden Retriever",
  "sexo": "hembra",
  "edadAnios": 3,
  "edadMeses": 0,
  "color": "Dorado"
}
```

Son obligatorios `nombre`, `especie` y `sexo`. Respuestas: 201 al crear y 200 al editar, con la Mascota. `GET /me/mascotas` regresa la lista sin paginar, con un máximo de 10 mascotas por cuenta.

`POST /me/mascotas/{id}/foto` usa `multipart/form-data` con el campo `foto`. Responde 200 con la Mascota y su nuevo `fotoUrl`. Errores: 400 `ARCHIVO_INVALIDO`.

Si la mascota es de otro usuario, la API responde 404 en lugar de 403, para no revelar que existe.

GET /me/mascotas/{id}/mensaje-whatsapp

Resuelve el texto de la pantalla del carnet: "se envía automáticamente en los mensajes de WhatsApp". Requiere `clinicaId` en query string.

Ejemplo: `GET /me/mascotas/3/mensaje-whatsapp?clinicaId=12`

```
{
  "ok": true,
  "data": {
    "texto": "Hola, Clínica Veterinaria Patitas. Los encontré en Vitalia.
Quiero información para mi mascota: Luna, perro hembra, Golden Retriever, 3
años, color dorado.",
    "url": "https://wa.me/5215598765432?text=Hola%2C%20Cl%C3%ADnica..."
  }
}
```

El front abre `url` directamente. Si la clínica no tiene WhatsApp: 400 `VALIDACION`; lo ideal es que el front oculte el botón cuando `whatsapp` es null.

Si el WhatsApp de la clínica tiene 10 dígitos, el backend le antepone la lada `52` al armar el link. El texto va codificado en la URL, así que acentos, espacios y símbolos como `&` llegan completos.

Sin sesión, el front arma su propio link `https://wa.me/<whatsapp>` con un saludo genérico; no necesita la API.

Endpoints de negocio para clínicas

Todos piden token con rol `clinica` y solo operan sobre clínicas donde el usuario es propietario; con otra clínica responden 403 `SIN_PERMISO`.

Método	Ruta	Para qué
GET	`/negocio/clinicas`	Mis clínicas, activas o no
POST	`/negocio/clinicas`	Registrar una clínica
GET	`/negocio/clinicas/{id}`	Ver mi clínica con datos internos
PUT	`/negocio/clinicas/{id}`	Editar datos generales
DELETE	`/negocio/clinicas/{id}`	Dar de baja
PUT	`/negocio/clinicas/{id}/horarios`	Reemplazar todos los horarios

Método	Ruta	Para qué
POST	`/negocio/clinicas/{id}/servicios`	Agregar servicio
PUT	`/negocio/clinicas/{id}/servicios/{servicioId}`	Editar servicio
DELETE	`/negocio/clinicas/{id}/servicios/{servicioId}`	Quitar servicio
POST	`/negocio/clinicas/{id}/fotos`	Subir foto
PUT	`/negocio/clinicas/{id}/fotos/orden`	Reordenar fotos
DELETE	`/negocio/clinicas/{id}/fotos/{fotoId}`	Borrar foto

Las respuestas de negocio usan ClínicaDetalle más dos campos: `activa` y `verificada`, para que el panel muestre si la clínica sigue en revisión.

POST /negocio/clinicas

Las coordenadas las obtiene el front cuando el dueño de la clínica pone el pin en el mapa del formulario.

```
{
  "nombre": "Clínica Veterinaria Patitas",
  "descripcion": "Atención integral para perros y gatos",
  "tipos": ["veterinaria"],
  "direccion": {
    "calle": "Av. Insurgentes 123",
    "colonia": "Del Valle",
    "alcaldiaMunicipio": "Benito Juárez",
    "ciudad": "Ciudad de México",
    "estado": "CDMX",
    "codigoPostal": "03100"
  },
  "lat": 19.3801,
  "lng": -99.1702,
  "telefono": "5598765432",
  "whatsapp": "5215598765432",
  "sitioWeb": null,
  "urgencias24h": false
}
```

Obligatorios: `nombre`, `tipos` con al menos uno, `direccion` completa, `lat`, `lng` y `telefono`. La clínica nace activa y con `verificada` en false; aparece en búsquedas desde ese momento.

Respuesta 201 con la clínica. `PUT /negocio/clinicas/{id}` recibe el mismo cuerpo y responde 200.

DELETE /negocio/clinicas/{id}

Es una baja lógica: la clínica deja de salir en búsquedas pero sus datos y reseñas se conservan. Responde 204.

PUT /negocio/clinicas/{id}/horarios

Reemplaza todos los horarios de golpe; así el front manda la semana completa como se ve en el formulario. Para una clínica 24 h se manda lista vacía y `urgencias24h` en true.

```
{
  "horarios": [
    { "diaSemana": 1, "abre": "09:00", "cierra": "14:00" },
    { "diaSemana": 1, "abre": "16:00", "cierra": "21:00" },
    { "diaSemana": 6, "abre": "10:00", "cierra": "15:00" }
  ]
}
```

Respuesta 200 con la lista guardada. Error 400 si `cierra` no es mayor que `abre` o si dos rangos del mismo día se enciman.

Servicios

`POST` y `PUT` reciben:

```
{ "categoriaId": 5, "nombre": "Cirugía", "descripcion": null, "precio":
2000.00, "esPrecioDesde": true }
```

Respuestas: 201 al crear y 200 al editar, con el Servicio. `DELETE` responde 204. Máximo 50 servicios por clínica.

Fotos

`POST /negocio/clinicas/{id}/fotos` usa `multipart/form-data` con el campo `foto`. Responde 201 con la Foto. La primera foto subida es la principal hasta que se reordenen.

`PUT /negocio/clinicas/{id}/fotos/orden` recibe `{ "ids": [88, 90, 89] }` en el orden deseado; el primero queda como foto principal. Responde 200 con la lista.

Errores: 400 `ARCHIVO_INVALIDO`; 409 `CONFLICTO` al pasar de 10 fotos.

Endpoints de admin

Todo lo de `/admin` pide rol `admin`. La moderación de la comunidad usa las rutas de borrado de la sección siguiente.

Método	Ruta	Rol	Para qué
GET	`/admin/clinicas`	Admin	Listar clínicas, filtrable por `verificada`
PATCH	`/admin/clinicas/{id}/verificacion`	Admin	Marcar o quitar verificación
PATCH	`/admin/clinicas/{id}/estado`	Admin	Activar o suspender una clínica
DELETE	`/admin/resenas/{id}`	Admin	Borrar una reseña ofensiva

GET /admin/clinicas

Parámetros: `verificada` true o false, `activa` true o false, `q`, `pagina` y `porPagina`. Responde lista paginada de ClínicaDetalle con `activa` y `verificada`.

PATCH /admin/clinicas/{id}/verificacion

```
{ "verificada": true }
```

Responde 200 con la clínica. La marca de verificada es la que respalda "proveedores verificados" en la pantalla de inicio.

PATCH /admin/clinicas/{id}/estado

```
{ "activa": false }
```

Responde 200. Una clínica suspendida deja de salir en búsquedas igual que una baja lógica.

DELETE /admin/resenas/{id}

Responde 204 y recalcula la calificación de la clínica.

Endpoints de la comunidad

Comunidad sustituye al blog: es el espacio donde los usuarios comparten opiniones y publicaciones. Leer es público; publicar y comentar pide sesión con cualquier rol.

Método	Ruta	Rol	Para qué
GET	`/comunidad`	Público	Publicaciones paginadas, la más reciente primero
GET	`/comunidad/{id}`	Público	Una publicación
POST	`/comunidad`	Logueado	Publicar
PUT	`/comunidad/{id}`	Autor	Editar mi publicación
DELETE	`/comunidad/{id}`	Autor o admin	Borrar la publicación y sus comentarios
GET	`/comunidad/{id}/comentarios`	Público	Comentarios paginados, el más antiguo primero
POST	`/comunidad/{id}/comentarios`	Logueado	Comentar
DELETE	`/comunidad/{id}/comentarios/{comentarioId}`	Autor o admin	Borrar un comentario

Las listas aceptan `pagina` y `porPagina` como el resto de la API. Si llega un token válido, la API llena `esMia` y `esMio`.

POST /comunidad y PUT /comunidad/{id}

```
{ "texto": "Luna ya tiene todas sus vacunas, gracias a Patitas por la atención" }
```

Respuestas: 201 al crear y 200 al editar, con la Publicación. Solo el autor edita; si no, 403 `SIN_PERMISO`. Al editar se llena `editadoEn`.

POST /comunidad/{id}/comentarios

Recibe `{ "texto": "..." }` y responde 201 con el Comentario. Si la publicación no existe: 404.

Borrar y moderar

El autor borra lo suyo y un admin puede borrar cualquier publicación o comentario. Ambos responden 204. Borrar una publicación borra también sus comentarios.

Fotos

Las publicaciones aceptarán hasta 4 fotos cuando se construya la subida de archivos, junto con las fotos de clínicas. Mientras tanto `fotos` siempre viene como lista vacía.

Acuerdos por equipo

Cada equipo marca sus puntos cuando los cumpla; así se ve rápido qué falta para integrar.

Frontend

- [ ] Consumir solo las rutas de este contrato, con el prefijo `/api/v1`.
- [ ] Decidir según `ok` y manejar errores por `codigo`, no por el texto de `mensaje`.
- [ ] Ante un 401, borrar el token y mandar al login.
- [ ] Obtener `lat` y `lng` con la geolocalización del navegador o el buscador del mapa y mandarlos en la búsqueda.
- [ ] Armar los links de "Cómo llegar" y "Compartir"; ocultar WhatsApp cuando el campo venga null.
- [ ] Usar los ejemplos JSON de este documento como datos de prueba mientras backend termina cada módulo.

Base de datos

- [ ] Crear tablas que respalden todos los modelos de datos, con nombres en snake_case.
- [ ] Guardar la ubicación de la clínica en las columnas `lat` y `lng` de tipo REAL, con un índice sobre ambas.
- [ ] Incluir `calificacion_prom` y `total_resenas` en la tabla de clínicas.
- [ ] Permitir varios horarios por día por clínica.
- [ ] Restringir una reseña por usuario por clínica.
- [ ] Entregar scripts SQL en dialecto SQLite en la carpeta `db/` del repo: creación, categorías iniciales y unas 20 clínicas de prueba en CDMX.

Infraestructura

- [ ] Ambiente con Python 3.12 o superior.
- [ ] Exponer el backend detrás de Apache en `/api`, con HTTPS en el ambiente público.
- [ ] Permitir cuerpos de hasta 6 MB para la subida de fotos.
- [ ] Dar un almacenamiento para fotos con URL pública.
- [ ] Monitorear `GET /api/v1/salud`.
- [ ] Cargar las variables de entorno de la tabla siguiente, sin subirlas al repo.

Variable	Ejemplo	Para qué
`APP_ENTORNO`	`produccion`	Nombre del ambiente
`APP_DEBUG`	`false`	Logs detallados; false en producción
`APP_ZONA_HORARIA`	`America/Mexico_City`	Cálculo de abierto ahora
`DB_RUTA`	`/var/vitalia/data/vitalia.db`	Ruta del archivo de la base SQLite
`JWT_SECRET`	secreto de 32 caracteres o más	Firma de tokens
`JWT_EXPIRA_HORAS`	`8`	Duración del token
`CORS_ORIGENES`	`["https://vitalia.mx"]`	Dominios del front permitidos
`FOTOS_RUTA`	`/var/vitalia/fotos`	Dónde se guardan las fotos
`FOTOS_URL_BASE`	`https://vitalia.mx/fotos`	URL pública de las fotos

Backend

- [ ] Implementar este contrato tal cual, con Swagger actualizado en `/docs`.
- [ ] Avisar a todos antes de cualquier cambio, por el proceso de la siguiente sección.
- [ ] Entregar cada módulo con pruebas automáticas pasando.
- [ ] Responder la búsqueda en menos de 500 ms con 1,000 clínicas cargadas.

Control de cambios

Ningún equipo cambia una ruta, un campo o un código de error sin pasar por estos pasos.

1. Quien necesita el cambio deja un comentario en este documento, sobre la parte que quiere modificar, explicando qué y por qué.
2. Backend revisa el impacto y responde en el mismo comentario.
3. Si afecta a otro equipo, ese equipo confirma en el hilo antes de seguir.
4. Backend actualiza este documento, Swagger y el historial de abajo.

Agregar un campo nuevo o un endpoint nuevo sube la versión menor, por ejemplo de 1.0 a 1.1. Quitar o renombrar algo que ya se usa rompe al front, así que solo se permite antes de la integración o creando `/api/v2`.

Versión	Fecha	Cambio
1.3	2026-11-12	Reseña agrega esMia y editadoEn; nota sobre la lada y la codificación del link de WhatsApp
1.2	2026-11-09	Se quitan los favoritos y el blog cambia por la comunidad
1.1	2026-11-03	Base de datos cambia de MySQL a SQLite, servidor web de Nginx a Apache y variables de BD a `DB_RUTA`
1.0	2026-11-02	Primera versión del contrato